assigns it to our node. The node sees that it has been allocated a web service task, and it launches a web container, meeting our desired state.

We can list the tasks in a stack with the following command:

```
$ docker stack ps myapp
ID   NAME                 IMAGE    …  DESIRED…  CURRENT…  …  …
qo…  myapp_db-migrator…   robise…  …  Shutdown  Complet…
ic…  myapp_database.1     postgr…  …  Running   Running…
8a…  myapp_redis.1        redis:…  …  Running   Running…
iw…  myapp_web.1          robise…  …  Running   Running…
```

The listing also tells us the current state of the task, whether that's successfully "Running," in the process of being started (such as "Pending" or "Preparing"), or in a number of other possible states.[10]

We can also list the tasks for a specific service. For example, to see the tasks running for our myapp_web service, we'd do:

```
$ docker service ps myapp_web
```

Scaling Up the Service

Currently, each of our services has been backed by a single task (and therefore container). We can instruct Swarm to increase or decrease the number of containers backing a service—known as scaling the service.

From the previous discussion, it should be clear how Swarm implements scaling. Imagine that the swarm receives a new definition of the web service, which specifies three containers. The orchestrator sees that only one task has been scheduled, and that therefore two more containers are required. It proceeds to create two new tasks, scheduling them across the available nodes in the cluster. The node (or nodes) allocate the tasks, then start up one container for each task they've been assigned. And voilà, we have scaled up our service to three containers.

OK, enough theory, let's see this in action.

Updating Our App

If we scale our app currently, it will be hard to tell the difference. How will we know that requests are being handled by different containers? Let's first make some changes to our app that will make this more obvious.

Edit app/controllers/welcome_controller.rb to look as follows:

10. https://docs.docker.com/engine/swarm/how-swarm-mode-works/swarm-task-states/